

An Area-Efficient HQC Accelerator With Unified Parallel Modules and Fine-Grained Scheduling

Jianxiong Zhang¹, Congwei Chen¹, *Graduate Student Member, IEEE*, Haolin Mai¹, Yanzhao Zhu¹,
Beihai Tan¹, Xiaoming Xiong¹, and Shuting Cai¹, *Member, IEEE*

Abstract—The Hamming quasicyclic (HQC) scheme has been selected by NIST for standardization in postquantum cryptography (PQC). The hardware implementations of HQC face challenges, including high latency, significant hardware overhead, and insufficient system-level scheduling optimization. To address these challenges, an area-efficient HQC hardware accelerator is proposed, in which the architecture of critical computational modules is optimized, and system-level scheduling is comprehensively enhanced. In particular, a unified and parallel sampling module (PSM) is first proposed. By incorporating only a small amount of detection logic, the sampling latency is significantly reduced. Moreover, the PSM supports both constant weight word (CWW) and fast nonbiased (FNB) modes. Subsequently, a novel, unified, four-way parallel polynomial arithmetic module (PPAM) is proposed. An innovative shared register bank design is proposed, which effectively reduces BRAM overhead. In addition, the PPAM supports both polynomial addition and multiplication modes. Finally, to address the challenge of insufficient system scheduling optimization, a fine-grained overlapped execution scheme (FGOES) is proposed, which further increases the execution overlap among modules and reduces the overall system latency. The proposed architecture is verified on an Xilinx Artix-7 field-programmable gate array (FPGA). Under the three NIST security levels, compared with state-of-the-art designs, the area-time product (ATP) is improved by $2.05\times$ to $11.02\times$, $2.33\times$, and $2.25\times$, respectively.

Index Terms—Code-based cryptography, field-programmable gate array (FPGA) implementation, Hamming quasicyclic (HQC), hardware security, key encapsulation mechanism (KEM).

I. INTRODUCTION

THE rapid advancement of quantum computing poses a severe threat to classical public-key cryptosystems, which rely fundamentally on the integer factorization and discrete logarithm problems [1]. To mitigate this security vulnerability, NIST initiated the postquantum cryptography (PQC) standardization process to rigorously evaluate and select robust PQC algorithms [2]. Consequently, lattice-based cryptographic schemes have been standardized as the primary standards [3]. Furthermore, the code-based Hamming quasicyclic (HQC)

scheme, distinguished by its unique mathematical foundation and rigorous security guarantees, has been selected and standardized as an alternative key encapsulation mechanism (KEM) [4], [5]. Driven by the increasingly stringent demands for real-time cryptographic processing in domains such as finance, cloud computing, and network communications, the efficient hardware implementation of the HQC algorithm has emerged as a prominent research focus [6], [7]. Although existing research has made notable progress in the architectural design and hardware implementation of HQC accelerators, achieving high area efficiency remains a significant bottleneck. In particular, the core computational modules that constrain the overall system performance present substantial room for architectural optimization and performance enhancement.

Challenge 1 (Suboptimal Intermodule Overlap): Prior architectures [8], [9] rely on strictly sequential execution or extensive module replication, leading to significant idle cycles and memory port contention in the arithmetic modules. To mitigate this issue, recent studies [10] overlap polynomial multiplication with hashing operations and reorder the sampling sequence of sparse polynomials to hide computation latency, thereby reducing the critical path delay [10], [11]. Nevertheless, existing scheduling architectures exhibit notable limitations. First, a unified scheduling framework supporting the full parameter set requires dynamic reconfiguration for varying configurations, introducing substantial multiplexing overhead and exacerbating shared memory port contention [10]. Second, the Decaps phase is bound by the implicit rejection mechanism, which mandates a complete reencapsulation operation. Consequently, static scheduling cannot further shorten the critical path while maintaining constant-time execution guarantees, hindering the joint optimization of throughput and resource utilization.

Challenge 2 (High Latency in Constant-Weight Sampling): The low-latency constant-time (LOCS) sampling module proposed in [12] leverages the Fisher–Yates shuffle algorithm to eliminate rejection sampling. Furthermore, the constant weight word (CWW) and fast nonbiased (FNB) schemes introduced in [8] provide fixed-schedule constant-weight sampling and FNB sampling, respectively, and have been adopted by HQC. Despite these algorithmic advancements, existing hardware implementations still face significant performance bottlenecks. In particular, the CWW scheme directly maps modular arithmetic onto digital signal processing (DSP) blocks, substantially increasing both the critical path delay and

Received 18 May 2026; revised 31 July 2026 and 20 August 2026; accepted 28 August 2026. This work was supported by the Key Area Research and Development Program of Guangdong Province under Grant 2022B0701180001. (Jianxiong Zhang and Congwei Chen contributed equally to this work.) (Corresponding authors: Beihai Tan; Shuting Cai.)

The authors are with the School of Integrated Circuits, Guangdong University of Technology, Guangzhou 510006, China (e-mail: bhtan@gdut.edu.cn; shutingcai@gdut.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2026.3729873

1063-8210 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: National Univ of Defense Tech. Downloaded on September 20, 2026 at 10:52:04 UTC from IEEE Xplore. Restrictions apply.

area overhead. Conversely, the FNB scheme necessitates large on-chip buffers to maintain a negligible failure probability, thereby incurring substantial memory overhead. Consequently, the sampling module remains a critical bottleneck, impeding the further development of fully hardware-accelerated HQC implementations that offer both high energy efficiency and ultralow latency.

Challenge 3 (High Area Overhead of Polynomial Operations): Existing literature predominantly employs shift-and-add architectures, reducing the computational complexity to $\mathcal{O}(n\omega)$. Current implementations primarily rely on BRAM-centric architectures. For instance, the designs in [9] and [13] employ address-marking logic to locate the nonzero coefficients of sparse polynomials, partition dense polynomials across BRAM arrays, and perform accumulation via XOR operations. Building upon this foundation, the architecture in [11] enhances parallelism by optimizing the shifters and implementing a multithread-port parallel accumulation structure. Furthermore, the work in [14] introduces a parallel segmented accumulation (PSA) technique for high-throughput scenarios, alongside a memoryless power permutation (PWP) architecture tailored for resource-constrained environments. Despite significant progress in architectural mapping reported in the literature, several critical limitations persist. First, BRAM-based designs are inherently constrained by the limited number of read/write ports, which bottlenecks the achievable parallelism. Aggressively instantiating multiple modules to boost parallelism incurs a substantial area overhead. Second, existing accelerators are predominantly optimized for fixed parameter sets, which restrict their adaptability across diverse mode configurations. Consequently, there is a pressing need for a novel sparse polynomial multiplication architecture that alleviates the memory port bottleneck, strikes an optimal balance between high throughput and minimal critical-path delay, and supports fixed-bitstream multiparameter operation.

To address these challenges, we present a set of optimization techniques, fully compliant with the latest HQC specification [4], to implement an area-efficient, dedicated HQC hardware accelerator on a field-programmable gate array (FPGA). Our primary contributions are summarized as follows.

- 1) Fine-grained overlapped execution scheme (FGOES), an optimized scheduling scheme tailored for HQC, is proposed. Furthermore, the sampling, encoding, and polynomial multiplication modules are pipelined to achieve an extensive overlap of their execution windows. Despite the incorporation of additional hash operations in the latest HQC specification, the proposed architecture achieves a latency reduction of 20.27%–26.31% over state-of-the-art designs [10], demonstrating superior hardware performance.
- 2) Parallel sampling module (PSM), a unified and highly efficient PSM for HQC, is proposed. By incorporating a small amount of additional detection circuitry, it achieves a significant reduction in sampling latency. Furthermore, it supports both the CWW and FNB operating modes, offering exceptionally high area efficiency.
- 3) Parallel polynomial arithmetic module (PPAM), a unified and area-efficient four-way PPAM for HQC, is

proposed. It can process four indices simultaneously in a single operation, and by addressing memory boundary alignment issues, it significantly reduces BRAM usage and improves hardware area utilization. Moreover, it supports both polynomial multiplication and addition modes. Compared with state-of-the-art implementations, the PPAM achieves up to a $4.91\times$ improvement in the area-time product (ATP).

We organize the remainder of this article as follows. Section II reviews the HQC algorithm. Section III presents the proposed hardware architecture and key modules. Section IV details the hardware scheduling optimization. Section V reports the implementation results and comparative analysis. Finally, we conclude in Section VI.

II. PRELIMINARIES

To bridge the gap between the HQC algorithmic specifications and the proposed hardware architecture, this section delineates the underlying mathematical foundations and base-line computational operations. Rather than providing a high-level overview, we explicitly detail the generation and utilization of critical polynomials, which directly dictate the dataflow and module design of our accelerator.

A. Algebraic Structure and Polynomial Arithmetic

HQC is constructed over the binary finite field $\mathbb{F}_2$ and the polynomial ring $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, where n is the smallest primitive prime greater than n_1n_2 . The core arithmetic operation is polynomial multiplication. For $u, v \in \mathcal{R}$, their product $w = u \cdot v$ is defined as follows:

$$w_k = \sum_{i+j \equiv k \pmod{n}} u_i v_j \quad \forall k \in \{0, 1, \dots, n-1\}. \quad (1)$$

In hardware, this cyclic convolution is equivalent to a sparse-dense polynomial multiplication when one operand is sparse (i.e., has a low Hamming weight). This operation constitutes the dominant computational bottleneck, motivating the design of our PPAM. Furthermore, a truncation operation $\text{Truncate}(\cdot, \ell)$ discards the trailing $\ell = n - n_1n_2$ bits to align with the concatenated code length.

B. Fixed-Weight Vector Sampling

The security and correctness of the HQC scheme are predicated on the sampling of random vectors with a fixed Hamming weight ω . According to the latest HQC specification, the protocol employs two distinct sampling modes: an unbiased FNB for key generation, and a constant-time CWW for encapsulation.

In particular, the sampling process generates a sequence of unique indices to construct the support of the sparse polynomial. Let ω denote the target weight. The $\text{SampleFixedWeightVect}_s$ algorithm employed in Algorithm 1 corresponds to the FNB mode, where the sampler draws random values from a pseudorandom number generator (PRNG), performs a reduction modulo n , and checks for collisions with previously sampled indices. If a collision

Algorithm 1 HQC.KEM KeyGen**Input:** None.**Output:** Encapsulation key ek_{KEM} , decapsulation key dk_{KEM} .

```

1:  $seed_{KEM} \leftarrow \{0, 1\}^{seed}$ 
2:  $(seed_{PKE}, \sigma) \leftarrow XOF(seed_{KEM})$ 
3:  $(seed_{PKE.dk}, seed_{PKE.ek}) \leftarrow l(seed_{PKE})$ 
4:  $ctx_{PKE.dk} \leftarrow XOF.Init(seed_{PKE.dk})$ 
5:  $y \leftarrow SampleFixedWeightVect_s(ctx_{PKE.dk}, \mathcal{R}_\omega)$ 
6:  $x \leftarrow SampleFixedWeightVect_s(ctx_{PKE.dk}, \mathcal{R}_\omega)$ 
7:  $ctx_{PKE.ek} \leftarrow XOF.Init(seed_{PKE.ek})$ 
8:  $h \leftarrow SampleVect(ctx_{PKE.ek}, \mathcal{R})$ 
9:  $s \leftarrow x + h \cdot y$ 
10:  $ek_{PKE} \leftarrow \{seed_{PKE.ek} \parallel s\}$ 
11:  $dk_{PKE} \leftarrow seed_{PKE.dk}$ 
12:  $ek_{KEM} \leftarrow ek_{PKE}$ 
13:  $dk_{KEM} \leftarrow \{ek_{KEM} \parallel dk_{PKE} \parallel \sigma \parallel seed_{KEM}\}$ 
14: return  $(ek_{KEM}, dk_{KEM})$ 

```

Algorithm 2 HQC.KEM Encaps**Input:** Encapsulation key ek_{KEM} .**Output:** Shared secret key K , ciphertext c_{KEM} .

```

1:  $m \leftarrow \{0, 1\}^{|K|}$ 
2:  $salt \leftarrow \{0, 1\}^{salt}$ 
3:  $(K, \theta) \leftarrow G(\{H(ek_{KEM}) \parallel m \parallel salt\})$ 
4:  $ctx_{PKE.ek} \leftarrow XOF.Init(seed_{PKE.ek})$   $\triangleright$  encrypt start
5:  $h \leftarrow SampleVect(ctx_{PKE.ek}, \mathcal{R})$ 
6:  $ctx_\theta \leftarrow XOF.Init(\theta)$ 
7:  $r_2 \leftarrow SampleFixedWeightVect(ctx_\theta, \mathcal{R}_{\omega_r})$ 
8:  $e \leftarrow SampleFixedWeightVect(ctx_\theta, \mathcal{R}_{\omega_e})$ 
9:  $r_1 \leftarrow SampleFixedWeightVect(ctx_\theta, \mathcal{R}_{\omega_r})$ 
10:  $u \leftarrow r_1 + h \cdot r_2$ 
11:  $v \leftarrow Encode(m) + Truncate(s \cdot r_2 + e, \ell)$ 
12:  $c_{PKE} \leftarrow \{u \parallel v\}$   $\triangleright$  encrypt end
13:  $c_{KEM} \leftarrow \{c_{PKE} \parallel salt\}$ 
14: return  $(K, c_{KEM})$ 

```

occurs, the value is discarded and resampled until exactly ω unique indices are obtained. The sparse polynomials generated via this method include the secret keys x and y during the KeyGen phase. Conversely, the `SampleFixedWeightVect` algorithm utilized in Algorithm 2 corresponds to the CWW mode. In this mode, the sampler draws random values from the PRNG, performs a reduction modulo $n - i$, and checks for collisions with previously sampled indices. In the event of a collision, a forced remapping is applied, and the sampling continues until ω unique indices are acquired. The sparse polynomials generated through this process correspond to the ephemeral random polynomials r_1 , r_2 , and e in the Encaps phase. This iterative collision-checking mechanism introduces severe data dependencies and incurs significant latency. To address this bottleneck, the proposed unified PSM is specifically tailored to accelerate this process.

C. HQC.KEM

The HQC.KEM is built upon an underlying public key encryption (PKE) scheme using the salted Fujisaki–Okamoto

transform with implicit rejection. The baseline computations involving the aforementioned polynomials are formalized as follows.

- 1) *Key Generation:* As shown in Algorithm 1, given a seed, the public key h (a dense polynomial) and the private keys x and y (sparse polynomials of Hamming weight ω) are derived. The intermediate public value s is then computed through polynomial multiplication and addition

$$s \leftarrow x + h \cdot y. \quad (2)$$

- 2) *Encapsulation:* As detailed in Algorithm 2, to encrypt a message m , ephemeral sparse polynomials r_1 and r_2 of Hamming weight ω_r , along with an error vector e of Hamming weight ω_e , are sampled. The encapsulated ciphertext $c_{PKE} = \{u \parallel v\}$ is then computed as follows:

$$u \leftarrow r_1 + h \cdot r_2 \quad (3)$$

$$v \leftarrow Encode(m) + Truncate(s \cdot r_2 + e, \ell). \quad (4)$$

These operations require multiple sparse-dense multiplications ($h \cdot r_2$, $s \cdot r_2$) and vector additions, heavily stressing the memory bandwidth and arithmetic units.

Algorithm 3 HQC.KEM Decaps**Input:** Decapsulation key dk_{KEM} , ciphertext c_{KEM} .**Output:** Shared secret key K' .

```

1:  $ctx_{PKE.dk} \leftarrow XOF.Init(seed_{PKE.dk})$ 
2:  $y \leftarrow SampleFixedWeightVect_s(ctx_{PKE.dk}, \mathcal{R}_\omega)$ 
3:  $v' \leftarrow v - Truncate(u \cdot y, \ell)$ 
4:  $m' \leftarrow Decode(v')$ 
5:  $(K', \theta') \leftarrow G(\{H(ek_{KEM}) \parallel m' \parallel salt\})$ 
6:  $c'_{PKE} \leftarrow HQC.PKE Encrypt(ek_{KEM}, m', \theta')$ 
7:  $c'_{KEM} \leftarrow \{c'_{PKE} \parallel salt\}$ 
8:  $\tilde{K} \leftarrow J(\{H(ek_{KEM}) \parallel \sigma \parallel c_{KEM}\})$ 
9: if  $m' = \perp$  or  $c'_{KEM} \neq c_{KEM}$  then
10:    $K' \leftarrow \tilde{K}$ 
11: end if
12: return  $K'$ 

```

- 3) *Decapsulation:* As shown in Algorithm 3, the receiver uses the secret key y to compute the shared secret. The core decryption involves computing the intermediate polynomial

$$v' \leftarrow v - Truncate(u \cdot y, \ell) \quad (5)$$

followed by `Decode(v')` to recover m . Subsequently, they perform reencryption as part of the implicit-rejection check. If the verification fails, a fallback key is returned.

The strict sequential dependencies among sampling x, y, r_1, r_2, e , executing the polynomial multiplications, and performing the encoding operations create significant pipeline stalls in conventional designs. This observation fundamentally motivates our FGOES to finely pipeline these baseline operations and maximize hardware utilization.

TABLE I
NOVELTY COMPARISON OF HQC HARDWARE ACCELERATORS

Feature	[8]	[11]	[9]	[13]	[14]	[10]	this work
Unified PSM	×	×	×	×	×	×	✓
· No-DSP reduction	Partial	×	×	×	×	✓	✓
Unified PPAM	×	×	×	×	×	✓	✓
· Shared reg. bank	×	×	×	×	×	×	✓
· Parallel degree	1	4	1	1	×	4	4
FGOES overlap	×	×	×	×	×	Partial	✓
All 3 security levels	×	×	×	×	×	✓	✓
Unified UPRNG	×	×	×	×	×	×	✓

✓: supported. ×: not supported. [14] has variable parallelism.

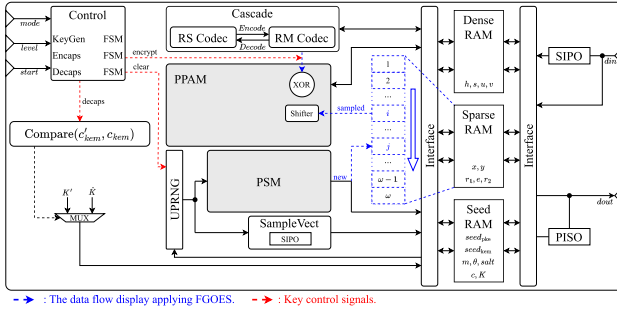


Fig. 1. Top-level system architecture of HQC.

D. HQC.KEM Versus ML-KEM

NIST selected HQC as a code-based complement to lattice-based ML-KEM, diversifying the hardness assumptions in its KEM portfolio. ML-KEM offers compact keys and ciphertexts (800–1568 bytes), whereas HQC incurs $3\times\text{--}10\times$ larger messages but avoids NTT-based modular polynomial multiplication. Its binary polynomial-ring arithmetic and fixed-weight sampling rely mainly on cyclic shifts and XOR operations over $\mathbb{F}_2$, enabling low-area, DSP-free hardware. HQC is therefore attractive when implementation cost and cryptographic diversity outweigh bandwidth. Our accelerator further reduces its area overhead, facilitating deployment alongside ML-KEM in resource-constrained systems.

III. HARDWARE DESIGN OVERVIEW

To achieve an area-efficient HQC accelerator, we jointly optimize area and latency. By integrating area-timing co-optimization for compact modules, utilization-maximizing resource sharing, and a custom dataflow scheduler that maximizes parallelism and minimizes the critical path, the proposed architecture delivers low latency with minimal area overhead. Table I highlights our architectural novelty against prior work.

A. System Architecture of HQC

To address the inherent challenges of HQC, particularly its computationally intensive nature and substantial memory requirements, we propose an area-efficient, fixed-bitstream HQC accelerator with control-driven mode and security-level selection. As illustrated in Fig. 1, the proposed top-level architecture comprises three core modules: the control component (left), the processing component (center), and the memory component (right). By adopting strategies such as a unified

modular design and functional partitioning, the architecture maximizes hardware resource utilization. Experimental results demonstrate that the unified architecture fully supports all three NIST security levels (HQC-1/3/5) while achieving a superior ATP compared to existing state-of-the-art designs.

The control component comprises three global finite state machines (FSMs) to manage concurrent task execution. This module efficiently schedules the KeyGen, Encaps, and Decaps operations to optimize performance. Notably, the novelty of the FGOES scheduling is demonstrated here, as detailed in Section IV.

Subsequently, the processing component is partitioned into three distinct functional modules.

- 1) The PSM (detailed in Section III-B), which supports configurable sampling strategies—specifically constant-time sampling and rejection sampling. It employs a dual-channel parallel architecture to accelerate the generation of sparse polynomial indices for x, y, r_1, e , and r_2 .
- 2) The PPAM (detailed in Section III-C), optimized for high-throughput execution of critical polynomial operations, such as $h \cdot y + x$ and $h \cdot r_2 + r_1$.
- 3) The unified PRNG (UPRNG) (detailed in Section III-D), which consolidates all requisite random number generation and seed expansion functionalities for HQC.KEM, including SHA3-256, SHA3-512, and SHAKE256 primitives.

Finally, the memory subsystem is organized into three logical storage categories interfaced with the arithmetic modules. To minimize BRAM utilization, we adopt a specialized memory-allocation strategy. The first category uses dedicated address regions to store fixed-length seed and auxiliary data. The second employs a single-address, dual-sparse-index organization to store the sparse polynomial indices required by different HQC operations. The third stores dense polynomial coefficients generated during polynomial computations, including the values used to construct the private and public keys. Each storage category may occupy multiple physical BRAM primitives according to its capacity and access requirements. Compared with the multi-RAM architecture in [10], this organization reduces the overall BRAM utilization by 35.71%.

This partitioning strategy is designed to maximize hardware resource sharing and computational parallelism. Notably, vector sampling and polynomial arithmetic collectively account for over 90% of the total execution time [4]. This computational bottleneck underscores the critical need for parallel architectural design and area-efficient optimization of these functional modules. Sections III-B–III-E elaborate on the specific implementation methodologies and hardware optimization details.

B. Parallel Sampling Module

Sampling is one of the key bottlenecks in the operational latency of HQC. The specification [4] defines two distinct sampling modes: 1) FNB for KeyGen, which eliminates statistical bias; and 2) CWW for encapsulation, using a fixed

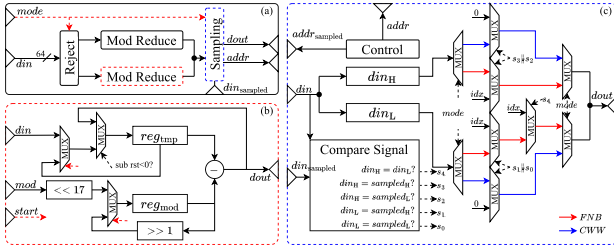


Fig. 2. Hardware architecture diagram of the PSM. (a) Overall architecture. (b) Modular reduction unit. (c) Sampling unit.

duplicate-check schedule. However, the hardware implementation poses two critical challenges: 1) a unified modular reduction design for different moduli and 2) eliminating early-termination-dependent cycle-count variation in the CWW duplicate-check control path. The work in [8] implements CWW and FNB architectures separately but incurs substantial resource overhead. For instance, it uses scarce DSP resources for modular reduction, which is unsuitable for deployment in resource-constrained scenarios, and it employs additional BRAM resources for sampling detection, significantly reducing resource reuse efficiency. To address these challenges, we propose a novel unified parallel sampling architecture, PSM. By decoupling the modular reduction operation from the sampling detection process, this architecture markedly improves hardware execution efficiency. Compared with [8], the proposed design achieves an 84.90%–87.28% area reduction.

Fig. 2(a) illustrates the hardware architecture of the PSM, which comprises two primary functional units: the modular reduction unit and the sampling unit. Dedicated control logic orchestrates the operational timing between these units and configures the architecture to support multiple operating modes.

1) *Modular Reduction Unit*: The sampling process requires the modular reduction of a 32-bit random integer by either n (in FNB mode) or $n - i$ (in CWW mode). This operation has been extensively investigated in prior literature. For instance, Deshpande et al. [8] propose a Barrett reduction-based scheme; however, to accommodate multiple moduli within a constant-time algorithm, the implementation leverages DSP blocks to perform Karatsuba multiplication, introducing substantial hardware overhead in resource-constrained scenarios. On the contrary, Gribok et al. [15] introduce a novel Barrett architecture that substitutes DSP resources with LUTs. Although this design yields a marginal improvement in the ATP metric compared to DSP-based approaches, it still incurs considerable LUT consumption. Recently, Antognazza et al. [11] employ a shift-subtract modular reduction unit that supports arbitrary moduli without DSP utilization, featuring low resource overhead at the cost of a multicycle pipeline architecture.

Fig. 2(b) illustrates the hardware architecture of the proposed modular reduction unit. The design employs a compact structure composed solely of subtractors, comparators, and shifters. Given its relatively low computational complexity and its position off the critical path, a nonpipelined architecture is adopted to minimize resource overhead. In this

design, each modular reduction operation completes in 19 clock cycles: Cycle 1 loads the 32-bit random number into the shift register. Iterative shift-and-subtract operations are executed during Cycles 2 through 18 [since the smallest HQC-1 modulus, 17 669 (15 bits), requires at most $32 - 15 = 17$ subtraction iterations]. Finally, Cycle 19 outputs the result. A pipelined architecture is deliberately avoided because the downstream sampling unit requires a variable number of cycles for repetition checks, owing to the nondeterministic nature of candidate acceptance. Consequently, a pipelined modular reduction unit would experience frequent pipeline stalls, rendering the throughput advantage of pipelining ineffective. Thus, this nonpipelined design choice is well-justified, yielding substantial hardware resource savings.

2) *Sampling Unit*: Each sampling operation necessitates a duplicate check against previously sampled values, causing the verification latency to scale quadratically with the number of samples. This relationship between the sample size and the operational latency is formally characterized by (6), where ω denotes the total number of samples. Furthermore, this formulation reveals a significant area overhead in the pipelined modular reduction architecture proposed in [11]. Since the duplicate check constitutes the primary bottleneck of the sampling module, the high-throughput modular reduction unit remains stalled until the verification is completed, thereby negating its computational speed advantage. Notably, the FNB architecture introduced in [8] allocates additional dedicated BRAM blocks to convert sparse indices into dense vectors, thereby facilitating duplicate checks. Although this strategy reduces the single-check latency to merely three clock cycles, it requires continuous monitoring of the bit-flip states at specific positions within the data block. This incurs substantial BRAM overhead, ultimately leading to excessive resource consumption and suboptimal memory utilization

$$t(\omega) = \frac{\omega(\omega + 1)}{2}. \quad (6)$$

To satisfy the throughput requirements of the PSM, the sparse-index memory is configured in a single-address, dual-value format. Moreover, the proposed parallel sampling unit involves multisignal comparison scenarios. As illustrated in Fig. 2(c), the parallel input architecture simultaneously compares four data values against one another, which differs fundamentally from conventional one-to-one comparison schemes.

For fixed public parameters, each CWW duplicate-check iteration completes the full duplicate scan without early termination; hence, the collision outcome does not change the cycle count of this control path. This property is limited to eliminating early-termination-dependent cycle-count variation in the CWW duplicate-check control path and does not establish general resistance to timing or other side-channel attacks.

Dual candidates are committed atomically using a single word-level write enable. Each sparse-memory word is organized as $MB_{spi}[a] = \{idx_H(a), idx_L(a)\}$, with idx_H and idx_L stored in the upper and lower 16-bit fields, respectively. In CWW mode, candidate collisions are deterministically resolved before the complete pair is written. In FNB mode,

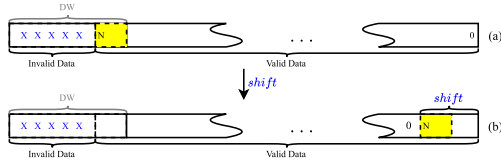


Fig. 3. Schematic of dense polynomial shifting. (a) Original dense polynomial. (b) Shifted dense polynomial.

the complete pair is discarded if either candidate fails a range or duplicate check or if the two candidates are equal. Thus, no collision can create a partially written sparse-memory word, and the word pointer advances only after a successful pair commit.

The PPAM maps the lower field id_{x_L} to the lower numbered shifter lane before the upper field id_{x_H} . For a four-index batch beginning at consumed-index count c , lane k is enabled only when $c + k < \omega$. The same validity signal qualifies both the shift launch and the corresponding accumulation-memory write enable. Therefore, for an odd weight, the unused field of the final physical word may be read but cannot affect the polynomial result. For example, when $\omega = 149$, the first 37 batches consume 148 indices, and the final batch enables only lane 0, which receives the valid lower field of the last sparse-memory word.

Owing to the parallel input architecture, the overall sampling latency represented by (7) is reduced to approximately 25% of the original execution time defined in (6)

$$t'(\omega) = \frac{\lceil \frac{\omega}{2} \rceil (\lceil \frac{\omega}{2} \rceil + 1)}{2} \approx \frac{1}{4} \frac{\omega(\omega + 1)}{2} \approx \frac{t(\omega)}{4}. \quad (7)$$

C. Parallel Polynomial Arithmetic Module

In HQC, polynomial multiplications dominate the overall computational latency. Existing hardware accelerators primarily adopt sparse-dense, BRAM-centric shift-and-accumulate architectures, achieving a theoretical complexity of $\mathcal{O}(\lceil n/DW \rceil \omega)$ and enhancing parallelism through multiple spatial instantiations. However, such designs still face critical limitations: 1) to maximize parallelism, the limited number of read ports in BRAMs forces designers to duplicate the memories storing the dense polynomial, resulting in suboptimal area efficiency [11], [14]; and 2) parameter-dependent control logic hinders the development of a unified datapath across different security levels [9], [16]. To address these issues, we propose a four-way PPAM. By introducing a shared register bank, the proposed PPAM leverages a single read port to simultaneously drive four shifters. This architecture completely eliminates the redundant duplication of dense polynomial memories, thereby significantly reducing the overall memory overhead. Compared with state-of-the-art polynomial multiplication modules, the PPAM achieves up to a $4.91\times$ improvement in the ATP. Furthermore, while maintaining the same degree of parallelism, it reduces the equivalent slice (EqS) count by 30.86%, demonstrating superior area efficiency for HQC polynomial acceleration.

In hardware implementations, rigorous boundary-condition handling is critical and encompasses two functional aspects:

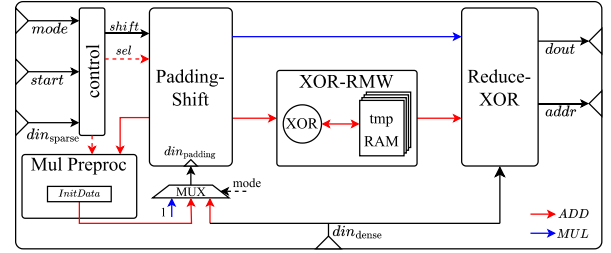


Fig. 4. Hardware architecture diagram of the PPAM.

1) *modular reduction boundaries*: managing the wraparound of sparse indices during computation; and 2) *memory word-length boundaries*: processing data that span memory words, which requires dedicated control because the polynomial degree is not divisible by the DW. As illustrated in Fig. 3(a) and (b), which depict the cyclic shift of a dense polynomial, the final (leftmost) memory block inherently contains invalid padding bits. Therefore, strict boundary control is required during the cyclic shift to prevent these invalid bits from propagating into the lower index valid-data region. The architectural mechanisms used to enforce these functional boundary conditions are detailed in the following sections.

We adopt sparse-dense polynomial multiplication as the foundational paradigm for the PPAM. Fig. 4 illustrates the proposed PPAM hardware architecture. By introducing a shared register bank, the design maximizes resource utilization while substantially reducing area overhead. The PPAM is configured with a 128-bit DW and supports four-way parallel processing. Architecturally, the module comprises three primary units: 1) the premultiplication control unit; 2) the padding-shift unit; and 3) the read-modify-write unit.

To facilitate the subsequent discussion, several key parameters are defined below and will be referenced throughout this section without further elaboration.

- 1) MB_{dpc} : A memory block designated for storing dense polynomial coefficients;
- 2) MB_{spi} : A memory block dedicated to storing sparse polynomial index values.
- 3) $R = n \bmod DW$: The length of valid data residing in the final address block of MB_{dpc} .
- 4) $Depth_{dpc} = \lceil n/(DW) \rceil$: The storage depth of MB_{dpc} ;
- 5) $shift_{old} = sparse_{index}[6:0]$: The initial shift offset, extracted from the least significant seven bits of the sparse polynomial index (where $7 = \log_2(DW)$).
- 6) $shift_{new} = (shift_{old} - R) \bmod DW$: A recalculated shift offset introduced to resolve boundary alignment constraints.
- 7) $glob = sparse_{index}[15:7]$: The starting address to which the shifted and truncated data is written.

1) *Premultiplication Control Unit*: This unit serves as a critical initialization module for polynomial multiplication. Because the HQC parameter n is prime and not an integer multiple of the DW, the final address block of MB_{dpc} inevitably contains invalid data. As illustrated in Fig. 3(b), when $shift_{old} > R$, the initial shifted data block must be extracted

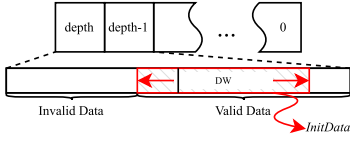


Fig. 5. Operational objective diagram of the premultiplication preprocessing control for polynomial multiplication.

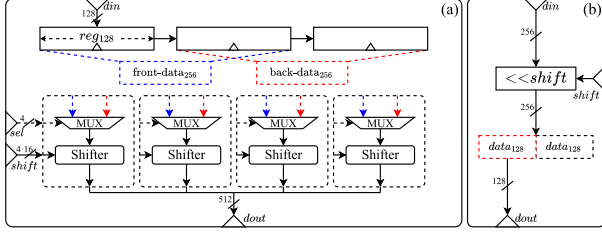


Fig. 6. Hardware architecture diagram of the padding-shift unit. (a) Overall architecture diagram. (b) Hardware architecture diagram of the shifter module.

from both the penultimate and last address blocks of MB_{dpc} . To address this memory word-boundary alignment issue, a dedicated pre-multiplication control unit is designed. In particular, before polynomial multiplication, the data residing in the last two address blocks of MB_{dpc} are concatenated and truncated. As depicted in Fig. 5, the primary function of this unit is to reconstruct and store a complete DW -bit data word, denoted as $InitData$.

To enhance resource utilization, the premultiplication control unit reuses the padding-shift unit. Although incorporating additional data multiplexers incurs a negligible area overhead, this design tradeoff is well justified by the substantial improvements in data alignment and overall computational efficiency.

2) *Padding-Shift Unit*: Serving as the core computational engine of the PPAM, this unit constitutes the primary architectural differentiator from prior work [8], [9], [11], [13]. To resolve the boundary misalignment issue during modular reduction, it concatenates data blocks from two consecutive addresses in MB_{dpc} and executes a noncyclic shift operation to extract the exact data blocks required for the subsequent accumulation. As illustrated in Fig. 6, this unit comprises a register bank consisting of three DW -bit registers and four $2DW$ -bit noncyclic shifters, which all take the register bank as a common input.

Upon the concurrent input of a set of sparse indices (i.e., four indices per set for a parallelism factor of 4) to initiate the operation, the input port din is initially loaded with $InitData$ to populate the register bank. Subsequently, data blocks are sequentially read from MB_{dpc} , starting at address zero and proceeding to the final block at address $Depth_{dpc}-1$, to feed the din port. This read sequence proceeds without pipeline stalls. At each successive clock cycle, the data within the register bank is shifted forward by one position as new data is input to din . Consequently, the register bank concurrently maintains the data blocks corresponding to three consecutive addresses in MB_{dpc} .

As illustrated in Fig. 6(a), the shifter selects either front-data₂₅₆ or back-data₂₅₆ as its $2DW$ data block input,

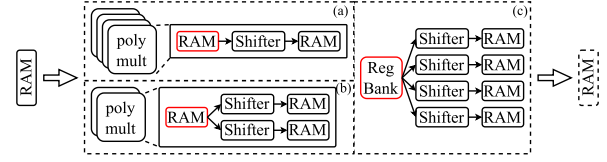


Fig. 7. RAM usage before and after optimization (with a parallelism factor of 4). (a) Simple dual-port RAM. (b) True dual-port RAM. (c) Shared register bank.

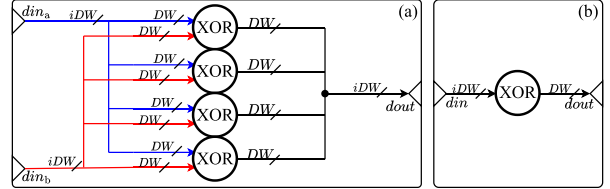


Fig. 8. XOR unit. (a) Vector XOR. (b) Reduce XOR.

controlled by the sel signal. During each processing round, as the read address for the external dense memory increments from 0 to $Depth_{dpc} - 1$, the global index $glob$, which serves as the write address for the temporary dense memory, must be incremented synchronously. However, since the initial value of $glob$ for a given $sparse_{index}$ is typically nonzero, $glob$ will inevitably wrap around to zero upon reaching $Depth_{dpc} - 1$ within a single round. This wrap-around introduces a boundary alignment issue during modular reduction, necessitating a one-cycle stall in the read operation. To handle this, the control unit determines whether a read stall is required; specifically, it decides whether to select back-data₂₅₆ as the $2DW$ input for the shifter to emulate the read operation during the stall. Prior work [13], [14] has addressed this issue by inserting a one-cycle bubble into the read pipeline; however, this approach inherently precludes further effective parallelization.

To handle the boundary conditions inherent in modular reduction, prior architectures are forced to insert stall cycles into read operations. Such read stalls are highly detrimental to polynomial multiplication modules that rely on pipelining and multiple instantiations to maximize parallelism. In particular, distinct $sparse_{index}$ values map to different $glob$ values, causing the boundary conditions to be triggered at unpredictable and irregular intervals. If a single read port is shared among multiple shifters, it will inevitably suffer from these unpredictable stalls, leading to severe pipeline hazards. To circumvent this issue, prior designs duplicate the dense polynomial memory to provide dedicated read ports for each shifter. As illustrated in Fig. 7(a) [8], [13] and (b) [10], [11], these architectures replicate multiple RAM blocks to increase the number of read ports, incurring substantial area overhead. On the contrary, Fig. 7(c) depicts our proposed shared register bank architecture, which employs multiplexers to eliminate memory duplication, thereby drastically reducing the area footprint with minimal overhead. This area efficiency becomes increasingly pronounced as the level of parallelism scales up.

3) *Read-Modify-Write Unit*: Fig. 8(a) illustrates the vector XOR unit, which is employed for internal polynomial

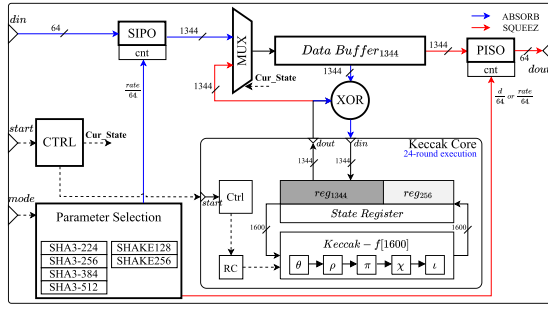


Fig. 9. Hardware architecture diagram of UPRNG.

multiplication operations. Each shifter is equipped with a temporary memory block, MB_{dpc} , implemented via BRAM. Since the dense polynomial memory is read-only during the computation, the intermediate results must be accumulated and temporarily stored in MB_{dpc} . Fig. 8(b) depicts the Reduce XOR unit, which operates during the output stage of the polynomial module to further accumulate data from the temporary MB_{dpc} and generate the final result.

Furthermore, the interface of the proposed PPAM architecture incorporates 16-bit n and 8-bit ω configuration inputs, which are asserted concurrently with the start signal. This feature unifies the architecture and allows a single fixed bitstream to support all three HQC parameter sets. Moreover, the degree of parallelism of the PPAM is fully parameterizable prior to logic synthesis. In summary, compared to state-of-the-art polynomial multiplication designs [8], [9], [11], [13], [14], [16], [17], the proposed PPAM significantly improves area efficiency.

D. Unified PRNG

The PRNG serves as a fundamental building block of the HQC scheme, tasked with generating cryptographically secure pseudorandom bits. While the initial versions of the HQC specification relied exclusively on SHAKE256, subsequent revisions have progressively integrated SHA3-256 and SHA3-512. To accommodate potential future updates to the HQC standard that may increase the reliance on additional SHA-3 variants, the UPRNG was designed from the outset to support the complete SHA-3 function set. This proactive architectural strategy effectively precludes the need for hardware redesign in the event of subsequent specification modifications.

Fig. 9 illustrates the hardware architecture of the proposed UPRNG module. The Keccak Core comprises a 1600-bit state register to perform the 24-round permutation. A 1344-bit buffer is placed outside the core to manage data input and output, corresponding to the maximum rate of 1344 bits in SHAKE128. This design effectively decouples the round function computations from the I/O operations, thereby enhancing the overall throughput.

As illustrated by the blue datapath in Fig. 9, the absorbing phase proceeds as follows. Transitioning from the idle state upon receiving the mode and start signals, the UPRNG enters the absorbing phase and asserts the o_need signal to accept incoming data. The SIPO unit is configured with a maximum count of $(rate)/64$ based on the selected mode. Once the

number of absorbed words reaches this threshold, the buffered data is transferred to the Keccak core, and the counter is reset to zero to resume reception. Note that for modes where the rate is less than 1344 bits, the upper bits of the buffer remain zero-padded, ensuring they do not interfere with the XOR operations. This process continues until the UPRNG receives the i_end signal, indicating the final data block, along with the $last_loc$ signal, which specifies the valid data boundary of this final block. Upon detecting these signals, the UPRNG appends the mode-specific domain separation bytes and transitions to the squeezing phase.

The red data flow in Fig. 9 illustrates the process of the squeezing phase. During squeezing, SHA3 outputs $[d/64]$ words and then returns to idle; only the required bits of the final SHA3-224 word are valid. SHAKE outputs $r/64$ words per rate block, performs another permutation, and repeats until external termination/reset is asserted. Thus, $capacity/64$ is not a PISO termination count.

E. RS and RM Codec

To ensure full system functionality, the Reed–Solomon (RS) codec leverages established architectures detailed in [18], [19], [20], [21], and [22], while the Reed–Muller (RM) codec adopts the design methodology of [8] and [23]. These blocks employ conventional design techniques and have been previously verified in complete HQC hardware implementations [9], [10]. Integrating these validated components with the proposed optimization blocks yields an area-efficient HQC accelerator. This modular integration guarantees complete system functionality while keeping the focus on our core architectural contributions.

IV. DATA FLOW SCHEDULING OPTIMIZATION

In the hardware accelerator design for HQC, dataflow scheduling remains a critical performance bottleneck, primarily rooted in the algorithm's inherent and stringent serial data dependencies. During the KeyGen and Encaps phases, the computational pipeline strictly follows a sequential execution order comprising sampling, polynomial multiplication, and encoding, wherein the sparse-dense polynomial multiplication dominates the overall execution time. This rigid dependency induces substantial pipeline bubbles and idle cycles, thereby impeding the effective overlapping of concurrent operations across different hardware modules. Prior scheduling schemes have attempted to mitigate this issue; for instance, the architecture proposed in [10] partially overlaps the sampling of x with the polynomial multiplication $y \cdot h$, effectively hiding the latency of a single sampling stage. Nevertheless, these approaches still suffer from substantial idle cycles and fail to fully exploit the available parallelism, as they typically operate under the conservative assumption that polynomial multiplication must wait until the entire sampling process is fully completed prior to its initiation.

A. Fine-Grained Overlapped Execution Scheme

To overcome the aforementioned limitations, the FGOES is proposed. Rather than attempting to eliminate data dependencies, which are inherent at the algorithmic level, the

TABLE II
SAMPLER RESOURCE UTILIZATION

Ref.	Support	Utilization					Freq. MHz	Cycles	Latency μs^b	ATP ^c
		LUT	FF	DSP	BRAM	EqS ^a				
Security strength 1 / HQC-1 ($\omega = 75$)										
This work	CWW ^d +FNB ^e	363	356	0	0.50	196.75	254	931	3.67	721.16(1x)
[12]	Fisher-Yates	1,560	766	0	2	814	170	976	5.74	4,673.32(6.48x)
[8](I)	CWW	201	229	4	1	800.25	201	3,062	15.23	12,190.87(16.90x)
[8](II)	FNB	316	124	0	2	503	223	1,479	6.63	3,336.04(4.63x)
[11]	CWW	1,055	1,189	0	2	687.75	230	2,335	10.15	6,982.16(9.68x)
[16]	FNB	646	330	0	6.50	1,539.50	183	851	4.65	7,159.10(9.93x)
[24]	CWW	1,046	1,370	0	0.50	367.50	279	3,037	10.89	4,000.35(5.55x)
Security strength 3 / HQC-3 ($\omega = 114$)										
This work	CWW+FNB	363	356	0	0.50	196.75	254	1,843	7.26	1,427.60(1x)
[12]	Fisher-Yates	1,553	761	0	3	1,024.25	185	1,636	8.84	9,057.69(6.34x)
[8](I)	CWW	211	245	5	1	937.25	200	6,817	34.09	31,946.17(22.38x)
[8](II)	FNB	295	125	0	2	497.75	236	2,226	9.43	4,694.88(3.29x)
[11]	CWW	1,010	1,178	0	2	676.50	237	5,027	21.21	14,349.22(10.05x)
[24]	CWW	1,027	1,289	0	0.50	362.75	279	6,820	24.44	8,867.22(6.21x)
Security strength 5 / HQC-5 ($\omega = 149$)										
This work	CWW+FNB	363	356	0	0.50	196.75	254	3,040	11.97	2,354.80(1x)
[12]	Fisher-Yates	1,569	779	0	3	1,028.25	181	2,268	12.53	12,884.37(5.47x)
[8](I)	CWW	216	248	5	1	938.50	204	11,487	56.31	52,845.83(22.44x)
[8](II)	FNB	314	192	0	2.50	608.50	242	3,248	13.42	8,166.98(3.47x)
[11]	CWW	1,027	1,172	0	2	680.75	225	8,834	39.26	26,727.76(11.35x)
[24]	CWW	1,049	1,360	0	0.50	368.25	279	11,510	41.25	15,191.96(6.45x)

^a: The computational formula presented in [10]. ^b: Latency(μs) = $\frac{\text{Cycles}}{\text{Freq. (MHz)}}$. ^c: EqS $\cdot \mu s$ [25]. ^d: Constant Weight Word. ^e: Fast and Non-Biased.

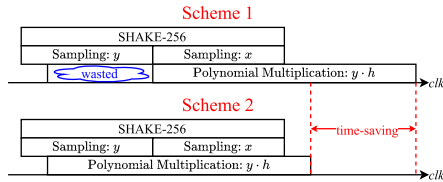


Fig. 10. FGOES.

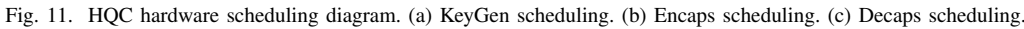
proposed scheme initiates polynomial multiplications as soon as a small batch of sparse indices becomes available, while the remaining indices are sampled concurrently. In particular, once four sparse indices—matching the four-way parallelism of the PPAM—are sampled and written into the MB_{spi} , the PPAM immediately launches the first batch of sparse-dense polynomial multiplications without waiting for all ω indices to be generated. As illustrated in Fig. 10, this technique effectively hides the majority of the sampling overhead and significantly enhances intermodule execution overlap.

A batch size of 4 is selected to match the four-shifter architecture of the PPAM, ensuring the full and parallel utilization of all shifters during each multiplication round. At dual-candidate iteration j , the PSM service interval is $\tau_{PSM}(j) = \max(19, j)$ cycles, whereas the PPAM consumes a batch of four indices every $\lceil n/128 \rceil$ clock cycles (e.g., 139 clock cycles for HQC-1). Since the generation throughput of the PSM exceeds the batch consumption rate of the PPAM,

the pipeline remains fully saturated. For larger values of ω , a greater proportion of the sampling latency can be masked, yielding a more significant relative latency reduction.

Furthermore, in the encapsulation algorithm, the encoded message $\text{Encode}(m)$ must be added to the result of a polynomial multiplication, as illustrated in (4). Previous designs [10], [17] execute the encoding operation independently and allocate a dedicated RAM block to store its output. The polynomial multiplication is then performed, followed by the polynomial addition only after the multiplication completes. This sequential approach incurs higher memory overhead and prevents the overlapping execution of different modules. On the contrary, our design concurrently executes the encoding operation and the polynomial multiplication, utilizing a hand-shake mechanism to stream the output data from the encoding module. During the output phase of the PPAM, the encoded data is synchronously added to the PPAM output by reusing the reduction XOR units within the PPAM. This strategy effectively reduces memory overhead and enables overlapped execution across modules.

- 1) Fig. 11(a) illustrates the hardware scheduling flow for the KeyGen algorithm. The blue box represents the polynomial multiplication stage, which employs FGOES for $y \cdot h$. By partially overlapping the y sampling stage, this scheme effectively masks most of the sampling latency, thereby significantly reducing the overall execution time.


$$T_{\text{Decaps-ours}} = t_{\text{polymul}}(\omega'') + \text{Decode} + T_{\text{Encaps-ours}} = 9393. \quad (15)$$

To ensure a fair comparison, all prior work considered in this study was implemented on the Xilinx Artix-7 FPGA platform, and the comparative data were directly extracted from their respective original publications. To the best of our knowledge, none of the existing HQC hardware designs report power or energy consumption metrics; therefore, a comparative power analysis is omitted. In cases where a prior work reported results under multiple configurations, we selected the highest-performing configuration for comparison.

The formula incorporates an additional round to account for the parallelized implementation of polynomial multiplication, which necessitates a final reduction of all intermediate partial sums during the output stage. Next, we model the overall execution latency of HQC, which is primarily dominated by the sampling, polynomial multiplication, and encoding and decoding operations. Targeting the HQC-1 parameter set ($n = 17\,669$, $\omega = 66$, $\omega_r = 75$, with encoding and decoding latencies set to the specific values reported in [11]),

TABLE III
POLYNOMIAL ARITHMETIC MODULE RESOURCE UTILIZATION

Ref.	Block	Parallelism	Platform	Utilization				Freq. MHz	Cycles	Latency	ATP ^c
	Size	Degree		LUT	FF	BRAM	EqS ^a			μs^b	
Security strength 1 / HQC-1 ($\omega = 66$)											
This work	128	4	Artix-7	3,096	1,865	8	2,470	213	2,519	11.83	29.21(1x)
[8]	64	1	Artix-7	620	245	2	579	270	18,482	68.45	39.63(1.36x)
[9]	128	1	Artix-7	2,239	612	4	1,407.75	198	10,921	55.16	77.65(2.66x)
[13]	128	1	Artix-7	2,443	662	4	1,458.75	201	10,501	52.24	76.21(2.61x)
[14](I) ^d	-	-	UltraScale+	20,334	4,942	18	8,899.50	403	1,410	3.50	31.14(1.07x)
[14](II) ^e	-	-	UltraScale+	53,041	53,457	0	13,260.25	471	4,500	9.55	126.69(4.34x)
[16]	128	1	Artix-7	1,741	878	4	1,283.25	289	9,307	32.20	41.33(1.41x)
Security strength 3 / HQC-3 ($\omega = 100$)											
This work	128	4	Artix-7	3,096	1,865	8	2,470	213	7,331	34.42	85.01(1x)
[8]	64	1	Artix-7	649	249	2	586.25	286	56,402	197.21	115.61(1.36x)
[13]	128	1	Artix-7	2,371	664	6	1,864.75	201	31,807	158.24	295.09(3.47x)
[14](I)	-	-	UltraScale+	18,905	4,181	18	8,542.25	376	4,200	11.17	95.42(1.12x)
[14](II)	-	-	UltraScale+	106,763	107,146	0	26,690.75	458	6,840	14.93	398.61(4.69x)
Security strength 5 / HQC-5 ($\omega = 131$)											
This work	128	4	Artix-7	3,096	1,865	8	2,470	213	15,367	72.15	178.20(1x)
[8]	64	1	Artix-7	644	223	2	585	283	118,426	418.47	244.80(1.37x)
[13]	128	1	Artix-7	2,330	669	8	2,278.50	211	71,869	340.61	776.08(4.36x)
[11]	128	4	Artix-7	4,962	2,333	11	3,572.50	155	15,054	97.12	346.97(1.95x)
[14](I)	-	-	UltraScale+	18,466	4,675	18	8,432.50	378	9,508	25.15	212.11(1.19x)
[14](II)	-	-	UltraScale+	172,896	173,363	0	43,224	442	8,940	20.23	874.26(4.91x)
[17]	128	1	Artix-7	3,913	491	0	978.25	218	60,058	275.50	269.50(1.51x)

^a: The computational formula presented in [10]. ^b: Latency(μs) = $\frac{\text{Cycles}}{\text{Freq. (MHz)}}$. ^c: $10^{-3} \cdot \text{EqS} \cdot \mu s$ [25]. ^d: BRAM-based. ^e: BRAM-less.

The definitions of the core polynomial arithmetic operations in the 2019 NIST PQC Round 3 specification remain identical to those in the 2025 specification [4]. However, the 2025 version introduces additional hash operations (i.e., SHA3-256 and SHA3-512), thereby increasing the total cycle count of the overall HQC execution. Therefore, module-level metric comparisons between this work and prior designs remain strictly valid. Furthermore, the system-level ATP improvement reported herein represents a conservative estimate relative to a hypothetical implementation of our design based on the 2019 specification. In addition, we adopt the EqS metric [10] for unified normalization, which comprehensively accounts for variations in the LUT ratio and BRAM configurations across different FPGA architectures.

Furthermore, to ensure design portability, we conducted an ASIC synthesis evaluation.

B. Sampling Performance Comparison

Table II compares the resource utilization and performance of the proposed PSM with those of prior standalone sampling modules across all three HQC security levels. The proposed unified PSM achieves the lowest ATP among all evaluated designs at every security level, while uniquely supporting both CWW and FNB sampling modes within a single compact module.

In terms of the ATP, the proposed PSM achieves values of 721.16, 1427.60, and 2354.80 for HQC-1, HQC-3, and HQC-5, respectively. These results correspond to ATP improvement factors ranging from 4.63 $\times$ to 16.90 $\times$ for

HQC-1, 3.29 $\times$ to 22.38 $\times$ for HQC-3, and 3.47 $\times$ to 22.44 $\times$ for HQC-5 over prior standalone sampling modules. This ATP advantage is most pronounced over designs employing DSP-based modular reduction [8](I) (up to 22.44 $\times$) and those featuring heavily pipelined architectures [11] (up to 11.35 $\times$). Such comparisons confirm that the proposed compact, DSP-free, and unified architecture is particularly well-suited for area-constrained PQC deployments.

It is worth noting that [16] requires slightly fewer cycles than the proposed PSM for HQC-1 (851 versus 931 cycles) by employing an early-termination duplicate-detection strategy, for which the duplicate-check latency may depend on the position at which a duplicate is detected. On the contrary, the proposed PSM completes the full duplicate scan in every CWW iteration. For fixed public parameters, this removes early-termination-dependent cycle-count variation from the CWW duplicate-check control path. The resulting 9.4% cycle overhead therefore represents the cost of eliminating this specific source of control-flow-dependent timing variation. This observation does not establish general resistance to timing or other side-channel attacks.

C. Polynomial Arithmetic Performance Comparison

As shown in Table III, the proposed PPAM achieves the lowest ATP among all evaluated designs across the three HQC security levels. In particular, on the Artix-7 platform, the ATPs of prior Artix-7 implementations are 1.36 $\times$ –4.36 $\times$ higher than that of the proposed PPAM. Furthermore, even when compared with UltraScale+-based designs [14] that benefit from an

TABLE IV
SYSTEM-LEVEL HQC RESOURCE UTILIZATION COMPARISON

Ref.	Platform	Utilization					Freq. MHz	KeyGen		Encaps		Decaps		ATP ^c
		LUT	FF	DSP	BRAM	Eq ^a		Cycles	μs^b	Cycles	μs	Cycles	μs	
Security strength 1 / HQC-1														
This work	Artix-7	22,702	12,350	0	18	9,491.50	194	4,767	24.57	9,112	46.97	12,361	63.72	1,283.80(1x)
[8](BAL) ^d	Artix-7	13,865	6,897	8	22	9,206.25	164	20k	121.95	30k	182.93	50k	304.88	5,613.57(4.37x)
[8](HS) ^e	Artix-7	15,214	7,293	8	24	9,967.50	178	20k	112.36	20k	112.36	40k	224.72	4,479.78(3.49x)
[9]	Artix-7	19,794	11,097	0	68	19,364.50	178	20k	112.36	40k	224.72	70k	393.26	14,142.61(11.02x)
[10]	Artix-7	26,561	13,636	0	28	12,576.25	143	5,577	39	11,726	82	18,304	128	3,131.49(2.44x)
[16]	Artix-7	20,201	13,685	0	62.5	18,300.25	157	12,355	78.69	17,885	113.92	32,512	207.08	7,314.51(5.70x)
[17]	Artix-7	26,802	17,430	8	32.5	14,666.50	162	5k	30.86	8k	49.38	16k	98.77	2,625.48(2.05x)
Security strength 3 / HQC-3														
This work	Artix-7	22,702	12,350	0	18	9,491.50	194	11,173	57.59	23,548	121.38	31,707	163.44	3,250.01(1x)
[10]	Artix-7	26,561	13,636	0	28	12,576.25	143	13,728	96	28,600	200	43,615	305	7,558.33(2.33x)
Security strength 5 / HQC-5														
This work	Artix-7	22,702	12,350	0	18	9,491.50	194	21,783	112.28	45,650	235.31	62,765	323.53	6,369.97(1x)
[10]	Artix-7	26,561	13,636	0	28	12,576.25	143	25,883	181	55,341	387	82,082	574	14,362.08(2.25x)

^a: The computational formula presented in [10]. ^b: Latency(μs) = $\frac{\text{Cycles}}{\text{Freq. (MHz)}}$. ^c: $10^{-3} \cdot \text{EqS} \cdot (\mu s_{\text{KeyGen}} + \mu s_{\text{Encaps}} + \mu s_{\text{Decaps}})$ [25]. ^d: Balance. ^e: High Speed.

inherent maximum frequency advantage of nearly $2\times$, the proposed PPAM achieves a highly competitive ATP. Notably, the ATPs of the BRAM-based and BRAM-less variants in [14] are $1.07\times$ – $1.19\times$ and $4.34\times$ – $4.91\times$ higher than that of the proposed PPAM, respectively.

By employing a four-way parallel architecture, the proposed PPAM reduces the cycle count per multiplication to 2519 for HQC-1. This corresponds to reductions of 72.9% relative to the fastest Artix-7 single-shifter implementation (9307 cycles [16]) and 86.4% relative to the design in [8] (18482 cycles). The standalone PPAM requires 3096 LUTs and eight BRAMs, resulting in an EqS value approximately $1.92\times$ that of the single-shifter design in [16]. Thus, the four-way architecture substantially improves throughput without incurring the $4\times$ area increase expected from direct replication. This sublinear area scaling is enabled by the shared register bank, which eliminates redundant replication of the dense-polynomial read memory and consequently achieves a lower ATP than the evaluated prior designs.

D. Overall Performance Comparison

Table IV summarizes the implementation results and provides a comparison with state-of-the-art HQC accelerators. Our proposed design consumes 22 702 LUTs, 12 350 FFs, zero DSP slices, and 18 BRAMs, resulting in a total of 9491.50 EqSs on the Artix-7 xc7a200t FPGA platform. The identical resource footprint across all three NIST security categories (i.e., HQC-1, HQC-3, and HQC-5) is a direct consequence of the unified PSM/PPAM/UPRNG architecture, which reuses the same datapath for all parameter sets via control-register-based mode configuration.

At the standalone-module level, the PSM is reported as 0.5 BRAM in Table II. At the system level, the sparse indices of x, y, r_1, e , and r_2 are stored in one shared BRAM through phasewise time multiplexing. Therefore, the sparse-index storage contributes one BRAM to the complete HQC accelerator. Together with eight BRAMs for the PPAM temporary

accumulation memories, eight for the system-level dense-polynomial memories, and one for auxiliary data, the total utilization is $8 + 8 + 1 + 1 = 18$ BRAMs.

Our design achieves the lowest BRAM utilization among the evaluated implementations, requiring only 18 BRAMs. This corresponds to reductions of 35.7%, 44.6%, and 73.5% compared with [10] (28 BRAMs), [17] (32.5 BRAMs), and [9] (68 BRAMs), respectively. These BRAM savings arise from two independent architectural innovations.

First, the shared register bank in the PPAM (Section III-C) eliminates the dense-polynomial BRAM replication required in prior multishifter architectures. Each MB_{dpc} (dense polynomial memory) occupies two BRAMs. In prior four-way parallel designs, two instances of MB_{dpc} are replicated to provide four independent read ports—one per shifter—consuming $2 \times 2 = 4$ BRAMs for the main dense-polynomial storage alone. Our shared register bank, by contrast, decouples read-port access from the shifters through a sliding-window mechanism, reducing the read-port requirement from four to one. Consequently, only a single MB_{dpc} instance (two BRAMs) is needed, saving two BRAMs at the main-memory level.

Second, the FGOES scheduling strategy (Section IV) executes the encoding operation concurrently with PPAM polynomial multiplication in the Encaps phase. The encoding output is streamed via a handshake mechanism and synchronously combined with the PPAM output by reusing the PPAM's internal reduction-XOR unit. This eliminates the dedicated encoding buffer RAM required in prior schemes [10], yielding additional BRAM savings beyond those achieved at the module level.

The ATP serves as a comprehensive figure of merit that jointly evaluates area efficiency and computational speed. At the HQC-1 security level, our design achieves the lowest reported ATP of 1.28×10^3 , representing a $2.05\times$ improvement over [17], a $2.44\times$ improvement over [10], a $3.49\times$ improvement over [8] (HS), and an $11.02\times$ improvement over [9]. These enhancements are primarily attributed

TABLE V
ASIC LOGIC SYNTHESIS SUMMARY (200 MHz, 110 nm CMOS)

Metric	TT (1.20 V, 25°C)	SS (1.08 V, 125°C)	FF (1.32 V, -40°C)
Logic Area (mm ²)	1.04	1.10	1.03
Memory Macro (mm ²)	4.55	4.55	4.55
Total Area (mm²)	5.59	5.66	5.58
Gate Count (kGE)	1,097.7	1,110.6	1,096.6
Worst Setup Slack (ns)	+0.00005	+0.00001	+0.00141
Setup Violations	0	0	0
Dynamic Power (mW)	122.43	96.10	166.27
Leakage Power	364.18 μ W	2.30 mW	134.73 μ W
Total Power (mW)	122.80	98.41	166.40
Cell Instances	101,463	104,129	101,183
Memory Macros	34	34	34

All corners meet setup timing at 200 MHz; area and power are post-synthesis estimates (DC V-2023.12-SP3); memory-macro area is fixed, and gate count is reported in NAND2-equivalent kGE. Power estimates assume vectorless 10% primary-input activity without workload VCD/SAIF annotation or silicon measurement and therefore does not establish absolute application energy efficiency.

to three synergistic factors: 1) the superior area efficiency of the unified modules, which are DSP-free and BRAM-efficient; 2) the maximized operating frequency; and 3) the minimized total latency achieved by the proposed FGOES architecture.

For HQC-3 and HQC-5, the only directly comparable work supporting all three security levels is [10]. The ATP of [10] is $2.33\times$ and $2.25\times$ larger than ours for HQC-3 and HQC-5, respectively. The slightly smaller ratio at the higher security level reflects the larger ω values, which increase the proportion of computation time that is parameter-dependent (e.g., duplicate detection) and thus less amenable to scheduling overlap.

E. ASIC Performance

To complement the FPGA implementation results and provide a technology-mapped assessment of the proposed HQC accelerator, we performed ASIC synthesis using a standard 110-nm CMOS process. The design was synthesized at a target clock frequency of 200 MHz (corresponding to a clock period of 5.0 ns) with practical I/O constraints, allocating 30% of the clock period to I/O delays and 10% to setup clock uncertainty. Synthesis was conducted across three standard process corners—typical (TT, 1.20 V/25 °C), slow (SS, 1.08 V/125 °C), and fast (FF, 1.32 V/-40 °C)—to verify timing closure and estimate postsynthesis power under varying process, voltage, and temperature (PVT) conditions. Table V summarizes the ASIC synthesis results across all three corners.

Postsynthesis results meet 200-MHz setup timing at all three PVT corners. Memory macros occupy 4.55 mm² (81.4% at TT), while the standard-cell area is 1.03–1.10 mm² (1097–1111 kGE). Under vectorless 10% input activity, total power is estimated at 98.41–166.40 mW; memory and clock networks contribute 23%–27% of total power and 41%–45% of dynamic power, respectively. Without workload-derived VCD/SAIF or silicon measurements, these results are preliminary and do not establish absolute application energy efficiency.

VI. CONCLUSION

We present an area-efficient, fixed-bitstream HQC accelerator that supports control-driven selection of the KEM operation and security level. The accelerator integrates a unified PSM, a four-way PPAM with a shared register bank, and FGOES. Implemented on an Artix-7 FPGA, it occupies 9491.5 EqS and achieves ATP improvement factors of $2.05\times$ – $11.02\times$ over the evaluated designs. The 110-nm ASIC synthesis results provide a preliminary technology-mapped assessment of the proposed architecture. The reported power values are vectorless estimates based on assumed switching activity and should not be interpreted as silicon measurements or as evidence of absolute application energy efficiency. For fixed public parameters, the duplicate-check control path completes the full scan without early termination, thereby eliminating cycle-count variation arising from this specific source. This property applies only to the duplicate-check control path and does not establish general side-channel resistance. Other potential sources of timing leakage, power and electromagnetic side channels, higher order attacks, and profiling attacks have not been evaluated; their assessment is left for future work. Source code is available at <https://github.com/ZzzJ1x/psm.git>

REFERENCES

- [1] P. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, Nov. 1994, pp. 124–134.
- [2] Q. D. Truong, H. Nguyen, T. Tan Nguyen, and H. Lee, “NIST post-quantum cryptography standards: A comprehensive review of theoretical foundations and implementations,” *IEEE Access*, vol. 14, pp. 14069–14097, 2026.
- [3] National Institute of Standards and Technology. (Aug. 2024). Module-Lattice-Based Key-Encapsulation Mechanism Standard. National Institute of Standards and Technology, *Federal Information Processing Standards Publication FIPS 203*. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [4] C. A. Melchor et al. (Aug. 2025). *Hamming Quasi-Cyclic (HQC). NIST Post-Quantum Cryptography Standardization Project*, Version Dated. Selected by NIST for Standardization. [Online]. Available: <https://pqc-hqc.org>
- [5] P. Rezaei, A. S. K. Rashid, A. A. Mand, and M. Saberikamarposhti, “Post-quantum key encapsulation mechanisms: A comparative review of ML-KEM, BIKE, and HQC,” in *Proc. 22nd IEEE Int. Colloq. Signal Process. Its Appl. (CSPA)*, May 2026, pp. 342–347.
- [6] M. Schöffel, J. Feldmann, and N. Wehn, “Code-based cryptography in IoT: A HW/SW co-design of HQC,” in *Proc. IEEE 8th World Forum Internet Things (WF-IoT)*, Oct. 2022, pp. 1–7.
- [7] A. Dolmeta et al., “TYRCA: A RISC-V tightly-coupled accelerator for code-based cryptography,” in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Mar. 2025, pp. 1–7.
- [8] S. Deshpande, C. Xu, M. Nawan, K. Nawaz, and J. Szefer, “Fast and efficient hardware implementation of HQC,” in *Selected Areas in Cryptography—SAC 2023 (Lecture Notes in Computer Science)*, vol. 14201. Cham, Switzerland: Springer, 2024, pp. 297–321.
- [9] C. Li, S. Song, J. Tian, Z. Wang, and Ç. K. Koç, “An efficient hardware design for fast implementation of HQC,” in *Proc. IEEE 36th Int. Syst. Chip Conf. (SOCC)*, Sep. 2023, pp. 1–6.
- [10] F. Antognazza, A. Barenghi, and G. Pelosi, “An efficient and unified RTL accelerator design for HQC-128, HQC-192, and HQC-256,” *IEEE Trans. Comput.*, vol. 74, no. 7, pp. 2306–2320, Jul. 2025.
- [11] F. Antognazza, A. Barenghi, G. Pelosi, and R. Susella, “A high efficiency hardware design for the post-quantum KEM HQC,” in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, May 2024, pp. 431–441.
- [12] P. He, Y. Tu, and J. Xie, “LOCS: LOW-latency and ConStant-timing implementation of fixed-weight sampler for HQC,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2023, pp. 1–5.

- [13] Y. Tu, P. He, Ç. K. Koç, and J. Xie, "LEAP: Lightweight and efficient accelerator for sparse polynomial multiplication of HQC," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 6, pp. 892–896, Jun. 2023.
- [14] P. He, Y. Tu, T. Bao, Ç. Ç. Koç, and J. Xie, "HSPA: high-throughput sparse polynomial multiplication for code-based post-quantum cryptography," *ACM Trans. Embedded Comput. Syst.*, vol. 24, no. 1, pp. 1–24, Jan. 2025.
- [15] S. Gribok, M. Langhammer, and B. Pasca, "FPGA modular multipliers using hybrid reduction techniques," in *Proc. 34th Int. Conf. Field-Programm. Log. Appl. (FPL)*, Sep. 2024, pp. 288–296.
- [16] Z. Wang, M. Zhou, X. Qiao, S. Zhuo, and W. Ge, "A high-performance hardware architecture for the Hamming quasi-cyclic key encapsulation mechanism," in *Proc. Conf. Sci. Technol. Integr. Circuits (CSTIC)*, Mar. 2026, pp. 1–3.
- [17] N. Pandey, S. Deshpande, D. D. Bohra, D. B. Roy, D. S. Banerjee, and J. Szefer, "Performance analysis of parameterizable HQC hardware architecture," in *Proc. 33rd Int. Conf. Sel. Areas Cryptogr. Sel. Papers (SAC)*, Ottawa, ON, Canada: Cham, Switzerland: Springer, Aug. 2026.
- [18] J. Massey, "Shift-register synthesis and BCH decoding," *IEEE Trans. Inf. Theory*, vol. IT-15, no. 1, pp. 122–127, Jan. 1969.
- [19] S. Scholl and N. Wehn, "Hardware implementation of a Reed–Solomon soft decoder based on information set decoding," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2014, pp. 1–6.
- [20] H. Fan and M. A. Hasan, "A survey of some recent bit-parallel GF(2ⁿ) multipliers," *Finite Fields Their Appl.*, vol. 32, pp. 5–43, Mar. 2015.
- [21] Y. Wu, "New scalable decoder architectures for Reed–Solomon codes," *IEEE Trans. Commun.*, vol. 63, no. 8, pp. 2741–2761, Aug. 2015.
- [22] X. Zhang, "VLSI architectures for Reed–Solomon codes: Classic, nested, coupled, and beyond," *IEEE Open J. Circuits Syst.*, vol. 1, pp. 157–169, 2020.
- [23] Y. Be'ery and J. Snyders, "Optimal soft decision block decoders based on fast Hadamard transform," *IEEE Trans. Inf. Theory*, vol. IT-32, no. 3, pp. 355–364, May 1986.
- [24] Y. Tu and J. Xie, "LEAH: Lightweight and efficient hardware accelerator for code-based PQC scheme HQC," in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, May 2026, pp. 34–44.
- [25] C. Chen et al., "A low-cost local masking radix-4 NTT against soft-analytical side-channel attacks," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 34, no. 3, pp. 1062–1066, Mar. 2026.



Haolin Mai received the B.E. degree from Dongguan University of Technology, Dongguan, China, in 2022. He is currently working toward the master's degree at the School of Integrated Circuits, Guangdong University of Technology, Guangzhou, China.

His research interests include hardware security and RISC-V architecture.



Yanzhao Zhu received the B.E. degree in electronic information engineering from Guangdong Ocean University, Zhanjiang, China, in 2025. He is currently working toward the master's degree at the School of Integrated Circuits, Guangdong University of Technology, Guangzhou, China.

His research interests include digital integrated circuit design.



Beihai Tan received the B.S. degree in information and computing sciences from Taiyuan University of Technology (TUT), Taiyuan, China, in 2002, and the Ph.D. degree in communication and information systems from South China University of Technology (SCUT), Guangzhou, China, in 2007.

In August 2010, he was a Postdoctoral Researcher and an Associate Research Fellow in electronic and information engineering at SCUT. He is currently a Professor with the School of Integrated Circuits, Guangdong University of Technology (GDUT),

Guangzhou. His research interests include algebraic coding theory, efficient decoding, and mobile computing.



Jianxiong Zhang received the B.E. degree in integrated circuit design and integrated systems from Guangdong University of Technology, Guangzhou, China, in 2024, where he is currently working toward the master's degree at the School of Integrated Circuits.

His current research interests focus on digital integrated circuit design.



Xiaoming Xiong received the B.Sc. degree from the Department of Radio, South China University of Technology, Guangzhou, China, in 1982, and the Ph.D. degree from the Department of Electrical Engineering and Communication, University of California, Berkeley, Berkeley, CA, USA, in 1988.

He is a Professor with Guangdong University of Technology, Guangzhou. His research interests include hardware/software cryptography, system-on-chip design, very large-scale integration design, electronic design automation, computer algorithms,

and graph theory.



Congwei Chen (Graduate Student Member, IEEE) received the B.E. degree in integrated circuit design and integrated systems from Guangdong University of Technology, Guangzhou, China, in 2024, where he is currently working toward the master's degree at the School of Integrated Circuits.

His research interests include hardware security, with specific interests in hardware accelerators for postquantum cryptography, side-channel attacks, and countermeasures.



Shuting Cai (Member, IEEE) received the B.Sc. and M.Sc. degrees in electronic engineering from Central South University, Changsha, China, in 2001 and 2004, respectively, and the Ph.D. degree in control science and engineering from Guangdong University of Technology, Guangzhou, China, in 2011.

He is a Professor at Guangdong University of Technology. His research interests include EDA and digital circuit design.